

Java Backend Architecture

Interview Series

Chapter 18.1

Nginx Architecture & Event Model

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 18.1 – Nginx Architecture & Event Model

Overview

Nginx (Engine-X) is a high-performance, event-driven web server and reverse proxy designed to handle tens of thousands of concurrent connections with minimal memory. Its architecture is fundamentally different from thread-per-connection servers like Apache: a single master process manages multiple worker processes, each running a non-blocking event loop. This makes Nginx the dominant choice for front-ending Java microservices, handling TLS termination, load balancing, caching, and rate limiting before requests reach the application tier.

Key Definitions

Term	Definition
Master Process	Reads and validates nginx.conf, binds ports, manages worker lifecycle. Zero-downtime reload: sends SIGUSR2 to spawn new workers.
Worker Process	Single-threaded event loop (epoll/kqueue). Each handles thousands of simultaneous connections non-blocking. worker_processes directive controls count.
epoll (Linux)	Linux kernel I/O event notification – O(1) scalability. Nginx uses edge-triggered mode for maximum throughput.
kqueue (BSD/macOS)	BSD/macOS equivalent of epoll. Nginx auto-selects the optimal mechanism per OS.
worker_connections	Maximum simultaneous connections per worker. Total capacity = worker_processes × worker_connections.
upstream block	Defines a named pool of backend servers with load-balancing algorithm and health check settings.
location block	Pattern-matched request routing rule. Priority: exact (=) > prefix no-regex (^~) > regex (~,~*) > prefix (/).
proxy_pass	Forwards HTTP request to an upstream backend. Core directive for reverse proxy operation.
Request Phases	11 phases: post-read, server-rewrite, find-config, rewrite, post-rewrite, preaccess, access, post-access, precontent, content, log.
ngx_http_*_module	Nginx is modular: static modules compiled in (e.g. ngx_http_proxy_module), dynamic modules loaded via load_module directive.

C10K problem: Apache struggles

Nginx: designed for C10K+

Code Examples

Basic nginx.conf Structure:

```
worker_processes auto; # one per CPU core worker_rlimit_nofile 65535; # match OS ulimit events {
worker_connections 4096; # per worker; total = auto * 4096 use epoll; # explicit (auto-selected on Linux)
multi_accept on; # accept all connections at once } http { sendfile on; # zero-copy file serving tcp_nopush
on; # batch TCP segments (with sendfile) tcp_nodelay on; # disable Nagle for keep-alive keepalive_timeout
65; keepalive_requests 1000; open_file_cache max=1000 inactive=20s; open_file_cache_valid 30s;
open_file_cache_min_uses 2; include /etc/nginx/conf.d/*.conf; }
```

Worker Process Tuning:

```
# Check available CPU cores $ nproc # e.g. 4 # nginx.conf worker_processes 4; # or 'auto'
worker_cpu_affinity auto; # pin each worker to a CPU (reduces cache miss) worker_priority -5; # slightly
higher OS priority # Verify active config $ nginx -T | grep worker # dump full config $ nginx -t # syntax
check only
```

Process Management Commands:

```
# Start / stop / reload nginx # start nginx -s stop # fast shutdown (SIGTERM) nginx -s quit # graceful
shutdown (drain connections) nginx -s reload # zero-downtime config reload (SIGHUP) nginx -s reopen #
reopen log files (log rotation) # Zero-downtime binary upgrade (hot upgrade) kill -USR2 $(cat
/var/run/nginx.pid) # spawn new master with new binary kill -WINCH $(cat /var/run/nginx.pid.oldbin) # drain
old workers kill -QUIT $(cat /var/run/nginx.pid.oldbin) # stop old master
```

Systemd Unit Tuning:

```
# /etc/systemd/system/nginx.service [Service] LimitNOFILE=65535 # file descriptor limit (must match
worker_rlimit_nofile) LimitNPROC=65535 Restart=on-failure RestartSec=5s
```

Configuration: Mime Types & Logging

```
http { include mime.types; default_type application/octet-stream; # Structured JSON access log for log
aggregation (ELK/Loki) log_format json_combined escape=json '{"time":'$time_iso8601','
'"remote_addr":'$remote_addr',' '"method":'$request_method',' '"uri":'$request_uri',' '"status":'$status','
'"bytes_sent":'$bytes_sent',' '"request_time":'$request_time','
'"upstream_response_time":'$upstream_response_time',' '"upstream_addr":'$upstream_addr}'; access_log
/var/log/nginx/access.log json_combined buffer=16k flush=5s; error_log /var/log/nginx/error.log warn; }
```

Nginx Module Architecture Comparison

Module Type	Examples	Load Method	Use Case
Core HTTP	ngx_http_core_module	Always compiled in	location, server, listen
Proxy	ngx_http_proxy_module	Always compiled in	proxy_pass, upstreams
SSL	ngx_http_ssl_module	Compiled in (--with-http_ssl_module)	HTTPS, TLS termination
Rate Limit	ngx_http_limit_req_module	Compiled in	limit_req, limit_req_zone
GeoIP2	ngx_http_geoip2_module	Dynamic: load_module ...geoip2.so	Country/city-based routing
Lua/OpenResty	ngx_http_lua_module	OpenResty distribution	Dynamic logic, WAF, JWT auth
Cache Purge	ngx_cache_purge	Dynamic/3rd-party	Cache invalidation API

Interview Q&A – Nginx Architecture & Event Model

Q1. What is the role of the Nginx master process?

The master process reads and validates `nginx.conf`, binds privileged ports (80/443), and manages worker process lifecycle. It never handles client connections directly. On `SIGHUP` (`nginx -s reload`), it re-reads config, starts new workers with updated config, and gracefully signals old workers to drain and exit — achieving zero-downtime reloads. On `SIGUSR2` (hot binary upgrade), it spawns a new master with the new binary while the old master keeps serving.

Q2. How does Nginx's event-driven model differ from Apache's threaded model?

Apache's prefork/worker MPM allocates one thread or process per connection. Under high concurrency (C10K+), this exhausts memory and causes excessive context switching. Nginx uses a single-threaded event loop per worker via `epoll` (Linux) or `kqueue` (BSD). A single worker handles thousands of connections by registering file descriptors with the kernel and reacting only when I/O is ready — $O(1)$ event dispatch, negligible per-connection overhead.

Q3. What is `epoll` and why does Nginx use edge-triggered mode?

`epoll` is Linux's scalable I/O event notification interface, replacing the $O(n)$ `select/poll`. Level-triggered (LT) mode fires events repeatedly while data is available. Edge-triggered (ET) mode fires once on state change, requiring the handler to read all data in a loop. Nginx uses ET for maximum efficiency: fewer syscalls, no redundant notifications. Edge-triggered demands non-blocking sockets, which Nginx enforces throughout.

Q4. What does `worker_processes auto` do and when would you override it?

`auto` sets worker count equal to CPU core count (from `sysconf(_SC_NPROCESSORS_ONLN)`), giving one worker per core to avoid context switching. Override to a lower number when Nginx shares the host with other CPU-intensive services, or to a higher number on I/O-bound workloads where workers frequently block on upstream responses (though proper async proxying avoids this).

Q5. How does zero-downtime config reload work?

`nginx -s reload` sends `SIGHUP` to the master. The master: (1) re-reads and validates new config; (2) opens new log files and listening sockets; (3) starts new worker processes with new config; (4) sends `SIGQUIT` to old workers, which stop accepting new connections and exit after all in-flight requests complete. During transition, both old and new workers run simultaneously — no dropped connections.

Q6. Explain the 11 Nginx request processing phases.

`post-read`: early request processing (e.g. `realip`). `server-rewrite`: `server-block` rewrites. `find-config`: location matching. `rewrite`: location rewrites, variable assignment. `post-rewrite`: internal redirect loop check (max 10). `preaccess`: `limit_conn`. `access`: `limit_req`, `auth_basic`, `allow/deny`. `post-access`: `satisfy all|any`. `precontent`: `try_files`, `mirror`. `content`: `proxy_pass`, `return`, static files. `log`: `access_log`. Modules hook into specific phases; understanding phases is critical for debugging unexpected request behavior.

Q7. What is the difference between `worker_connections` and the maximum concurrent connections?

`worker_connections` is the limit per worker process. Maximum concurrent connections = `worker_processes` x `worker_connections`. However, each proxied connection uses 2 file descriptors (client + upstream), so effective proxy connections = $(\text{worker_processes} \times \text{worker_connections}) / 2$. Also constrained by OS file descriptor limit (`ulimit -n / worker_rlimit_nofile`).

Q8. How does Nginx handle upstream connections efficiently?

Nginx maintains `keepalive` connection pools to upstream servers via the `keepalive` directive in upstream blocks. `proxy_http_version 1.1` and `proxy_set_header Connection ''` enable HTTP/1.1 keepalives. The `keepalive N` directive caches N idle connections per worker. This avoids TCP handshake overhead for every request to Java backends, critical for high-throughput microservices.

Q9. What is the `open_file_cache` directive and why does it matter for static serving?

`open_file_cache` caches file descriptors, `stat()` results, and directory lookups in memory. Without it, every static file request triggers `open()/stat()` syscalls. Parameters: `max` (cache size), `inactive` (evict if not accessed within), `valid` (re-check period),

min Uses (minimum accesses before caching). Dramatically reduces syscall overhead for Java apps serving many static assets.

Q10. How does sendfile improve static file serving performance?

sendfile(2) is a Linux syscall that transfers file data from disk directly to a socket buffer in kernel space, bypassing the user-space copy (read() + write()). Nginx's sendfile on enables this. Combined with tcp_nopush on (buffers response headers until sendfile data ready, reducing TCP segments) and tcp_nodelay on (disables Nagle for keepalive), this is the standard high-performance static serving configuration.

Q11. What happens during a hot binary upgrade?

kill -USR2 : master renames PID file and exec()s new binary, spawning new master. Old master is still running. New master starts new workers. Verify new workers are healthy: kill -WINCH (old workers drain). If OK: kill -QUIT (old master exits). Rollback: kill -HUP (re-read config). The entire process involves no dropped connections.

Q12. Explain Nginx's memory model for connection handling.

Each active connection allocates a connection object from a pre-allocated connection pool (worker_connections). Nginx uses memory pools (ngx_pool_t) for request-scoped allocations that are bulk-freed at request end — avoiding malloc/free fragmentation. Buffer sizes are configurable: client_header_buffer_size (initial), large_client_header_buffers (overflow). This makes Nginx's per-connection memory overhead ~1-2KB vs Apache's per-thread ~8MB stack.

Q13. How does multi_accept on affect worker behavior?

By default, a worker accepts one new connection per epoll event. multi_accept on makes the worker accept all pending connections in the accept queue before returning to event processing. This reduces latency under burst traffic. Caution: can starve existing connections during extreme spikes. Generally beneficial for high-traffic Nginx deployments fronting Java microservices.

Q14. What is the difference between nginx -s stop and nginx -s quit?

nginx -s stop sends SIGTERM: workers immediately terminate all connections and exit, even if requests are in flight — fast but potentially drops active requests. nginx -s quit sends SIGQUIT: workers stop accepting new connections, allow all in-flight requests to complete, then exit gracefully. Always use quit for production shutdowns to avoid dropped requests to Java backends.

Q15. How does Nginx handle SSL/TLS handshakes in its event model?

TLS handshakes are non-blocking using OpenSSL with BIO abstractions. After TCP accept, Nginx registers the socket for read events and performs the TLS handshake incrementally across multiple epoll events. ssl_session_cache shared:SSL:10m stores session tickets/IDs across workers (shared memory). ssl_session_timeout 1d. This allows a single worker to manage many concurrent TLS handshakes without blocking.

Q16. What is the significance of worker_cpu_affinity?

worker_cpu_affinity pins each worker to specific CPU cores via sched_setaffinity(). This prevents workers from migrating between cores, improving CPU cache locality (L1/L2 hit rates for connection pools and memory pools). worker_cpu_affinity auto assigns sequentially. Significant performance gain on NUMA systems where cross-node memory access is expensive.

Q17. Explain how Nginx implements upstream health checks.

Nginx open-source only supports passive health checks: max_fails (failures within fail_timeout window marks server down), fail_timeout (window duration and down period). Nginx Plus adds active health checks (health_check directive: periodic probe, customizable conditions). Open-source alternative: nginx_upstream_check_module (3rd-party). For Java microservices, combine with Kubernetes readiness probes to remove unhealthy pods from upstream before Nginx detects failure.

Q18. What are Nginx variables and how are they evaluated?

Nginx variables (prefixed \$) are evaluated lazily at request time. Types: built-in (\$uri, \$remote_addr, \$status, \$upstream_addr), configuration-set (set \$var value), module-provided (\$geoip2_city, \$limit_rate). Variables enable dynamic configuration: proxy_pass http://\$http_host\$request_uri (dynamic routing). map module transforms one variable to another efficiently (O(1) hash lookup). Avoid using variables in locations — prevents location caching.

Q19. How does Nginx's upstream module implement load balancing?

upstream block defines a server pool. Algorithms: round_robin (default, sequential with weights), least_conn (fewest active connections, best for variable-duration Java API calls), ip_hash (client IP hashed to consistent server, session affinity), hash \$key (arbitrary key hash, consistent hashing available in Plus). Each algorithm is a C module hooking into the upstream peer selection phase.

Q20. What is the try_files directive and how does it work?

try_files checks file system paths in order, serving the first that exists, falling back to a named location or URI. Example: try_files \$uri \$uri/ @backend — checks if file exists, then directory index, then proxy to @backend named location. The final argument is always a fallback (URI or named location starting with @). Essential for single-page applications and Java apps with separate static and dynamic content.

Q21. How does Nginx handle HTTP/2?

listen 443 ssl http2 enables HTTP/2. Nginx terminates HTTP/2 from clients (multiplexed streams, header compression via HPACK, server push) and proxies to Java backends over HTTP/1.1 or HTTP/2 (proxy_http_version 2 with ngx_http_v2_module). HTTP/2 eliminates head-of-line blocking at the HTTP layer. For gRPC backends: grpc_pass with http2 required.

Q22. What is the resolver directive and when is it needed?

resolver specifies DNS server for dynamic upstream resolution. Required when proxy_pass uses a variable (proxy_pass http://\$upstream) — Nginx resolves at request time. Without resolver, hostnames in proxy_pass are resolved once at startup. resolver 8.8.8.8 valid=30s; — cache DNS for 30s. Critical for service mesh integrations where upstream addresses change dynamically (Kubernetes service DNS).

Q23. Explain the satisfy directive.

satisfy controls access module logic. satisfy all (default): ALL access modules must permit (auth_basic AND allow/deny). satisfy any: ANY access module permit is sufficient (auth_basic OR allow from internal IP). Useful when internal IPs bypass authentication but external IPs require basic auth. Example: internal monitoring bypasses JWT auth that external Java API clients require.

Q24. How does Nginx implement request buffering and when should you disable it?

proxy_buffering on (default): Nginx buffers the entire upstream response before sending to client, releasing the upstream connection faster. proxy_buffer_size (response headers) and proxy_buffers (body buffers). Disable with proxy_buffering off for: server-sent events (SSE), long-polling, streaming responses from Java/Spring WebFlux. With buffering off, Nginx acts as a transparent pass-through, increasing upstream connection hold time.

Q25. What are named locations and how are they used?

Named locations (location @name {}) cannot be accessed directly by clients — only via try_files, error_page, or internal redirects. They bypass the normal location matching sequence. Common patterns: error_page 503 @maintenance (serve maintenance page on 503), try_files \$uri @app (fall through to app), recursive internal redirects for A/B routing logic.

Q26. How does the limit_req module implement token bucket rate limiting?

limit_req_zone \$key zone=name:size rate=N r/s defines a shared memory zone with a leaky bucket. limit_req zone=name burst=B nodelay: burst allows B requests to queue; nodelay processes queued requests immediately (no artificial delay), returning 429 only when burst is exhausted. Without nodelay, requests are delayed to match the rate. Zone key \$binary_remote_addr uses 4-byte IPv4 (vs \$remote_addr 7-15 bytes) for 2x memory efficiency.

Q27. Explain Nginx's shared memory zones.

Shared memory (mmap) is accessible by all worker processes without IPC overhead. Used by: limit_req_zone (rate limiting state), limit_conn_zone (connection counting), proxy_cache_path (cache metadata), ssl_session_cache (TLS session reuse), upstream keepalive (Plus). The ngx_slab_pool allocator manages shared memory. Lock contention can become a bottleneck at extremely high request rates — monitor with nginx_status.

Q28. What is the geo module and how does it enable IP-based routing?

geo \$variable { default value; CIDR value; } creates a variable based on \$remote_addr CIDR matching. O(1) lookup using radix tree. Use cases: geo \$is_internal { default 0; 10.0.0.0/8 1; 192.168.0.0/16 1; } to bypass rate limiting for internal IPs. Combined with map and proxy_pass for geographic routing. GeoIP2 module provides country/city data from MaxMind databases.

Q29. How does Nginx handle upstream connection failures and retries?

proxy_next_upstream error timeout invalid_header http_500 http_502 http_503: retry on these conditions. proxy_next_upstream_tries N: maximum retries. proxy_next_upstream_timeout T: total retry budget. Caution: only safe for idempotent requests (GET/HEAD) — POST retries can cause duplicate side effects in Java backends. proxy_connect_timeout (TCP connect) vs proxy_read_timeout (between upstream data packets).

Q30. What is the mirror module and how is it used?

mirror creates a sub-request to a mirror URI for every original request, useful for shadow testing. mirror /mirror_backend; mirror_request_body on. The mirror request is fire-and-forget — response is discarded. Use case: route 100% of production traffic to both current and new Java backend versions simultaneously to compare behavior without affecting users.

Q31. Explain how Nginx implements HTTP keep-alive connections with clients.

keepalive_timeout 65s: server sends Keep-Alive header and holds connection for 65s. keepalive_requests 1000: maximum requests per keepalive connection before close. keepalive_disable msie6: disable for broken clients. HTTP/1.1 keepalive is default; HTTP/1.0 requires Connection: keep-alive header. Reduces TCP handshake overhead for browsers/API clients making multiple requests.

Q32. What is the sub_filter module?

sub_filter old_string new_string replaces text in proxied responses. sub_filter_once off enables all occurrences. sub_filter_types text/html text/javascript. Use case: rewrite absolute URLs in legacy Java app responses when path prefix changes, inject analytics scripts into HTML without modifying backend. Requires proxy_buffering on (needs full body to search).

Q33. How does Nginx handle large file uploads?

client_max_body_size 100m: maximum request body size (default 1m; 0=unlimited). client_body_buffer_size 128k: buffer in memory before spooling to disk (client_body_temp_path). client_body_timeout: time between successive reads. For Java file upload services, set proxy_request_buffering off to stream the request body directly to upstream without buffering in Nginx — avoids double storage and reduces latency for large uploads.

Q34. What is the stub_status module and what metrics does it expose?

location /nginx_status { stub_status; } exposes: Active connections (currently open), accepts (total accepted), handled (total handled = accepts for no drops), requests (total), Reading (request headers being read), Writing (response being sent), Waiting (keepalive idle). Prometheus exporters (nginx-prometheus-exporter) scrape stub_status. Nginx Plus exposes richer JSON API with upstream health, zone stats, cache stats per endpoint.

Q35. Explain how Nginx proxy_cache works.

proxy_cache_path defines storage: path, levels (directory depth), keys_zone (shared memory for keys), max_size (evict LRU when exceeded), inactive (evict if not accessed). proxy_cache zone_name enables caching in a location. proxy_cache_key defines cache key (default \$scheme\$proxy_host\$uri\$is_args\$args). proxy_cache_valid 200 10m caches 200 responses for 10 minutes. X-Cache-Status header (add_header \$upstream_cache_status) shows HIT/MISS/BYPASS/EXPIRED.

Q36. How does Nginx implement SSL session resumption?

Two mechanisms: session IDs (server stores session in ssl_session_cache shared:SSL:10m, client presents ID on reconnect — avoids full handshake) and session tickets (server encrypts session state in ticket sent to client, stateless — scales across Nginx workers/servers but requires ssl_session_ticket_key rotation for forward secrecy). ssl_session_timeout 1d. Critical for reducing TLS handshake overhead to Java APIs.

Q37. What is the difference between return and rewrite directives?

return code [URL] sends HTTP redirect/response immediately, stopping all processing. Faster — no regex evaluation. rewrite regex replacement [flag] transforms the URI using PCRE regex and continues processing (or redirects with last/break/redirect/permanent flags). return preferred for simple redirects (301/302), rewrites for complex URI transformations. return 301 is processed at server phase; rewrite at rewrite phase.

Q38. How does Nginx handle WebSocket proxying?

WebSocket requires HTTP/1.1 upgrade. Config: proxy_http_version 1.1; proxy_set_header Upgrade \$http_upgrade; proxy_set_header Connection 'upgrade'; Nginx then transparently tunnels the bidirectional WebSocket frames. proxy_read_timeout must be long enough (e.g. 3600s) to sustain idle WebSocket connections. proxy_send_timeout similarly. Required for Spring WebSocket / SockJS Java backends.

Q39. What is the map module and how does it enable dynamic configuration?

map \$source_variable \$target_variable { default value; pattern value; } creates a hash-based lookup table (O(1)). Evaluated lazily. Examples: map \$request_method \$no_cache { POST 1; default 0; } for cache bypass. map \$host \$upstream { api.example.com api_servers; app.example.com app_servers; } for virtual host routing. Avoids if-in-location performance anti-pattern.

Q40. How do you implement A/B testing or canary releases with Nginx?

split_clients module: split_clients \$remote_addr \$variant { 10% 'canary'; * 'stable'; } uses MurmurHash2 on the key, deterministically assigns. upstream \$variant routes to different pools. Alternative: Nginx Ingress Controller canary annotations (nginx.ingress.kubernetes.io/canary: 'true', canary-weight: '10') for Kubernetes deployments. Sticky sessions via ip_hash maintain consistent assignment per client.

Q41. Explain the if directive limitations in Nginx.

if is evil inside location blocks due to Nginx's non-linear configuration inheritance. Safe uses: if in server context (return, rewrite only). Problematic: if creates implicit nested location with inherited directives, causing unexpected behavior. Use map module for variable-based logic, try_files for existence checks, geo for IP logic. The Nginx wiki explicitly warns: 'if is evil when used inside location'.

Q42. What is proxy_hide_header and when is it used?

proxy_hide_header suppresses specific response headers from upstream before forwarding to client. Default hidden: Date, Server, X-Powered-By. Use proxy_hide_header X-Internal-Service-Version to prevent backend version disclosure (security hardening). proxy_pass_header re-enables headers suppressed by default. add_header adds custom headers to client response. more_headers module (headers-more-nginx-module) for advanced header manipulation.

Q43. How does Nginx handle gzip compression?

gzip on; gzip_comp_level 6 (1-9, balance CPU vs compression); gzip_min_length 1000 (skip tiny responses); gzip_proxied any (compress proxied responses); gzip_types text/plain text/css application/json application/javascript. gzip_vary on adds Vary: Accept-Encoding. For Java REST APIs returning JSON, gzip typically achieves 60-80% size reduction. gzip_static on serves pre-compressed .gz files if available.

Q44. What is the Nginx Plus active health check and how does it differ from open-source?

Nginx Plus health_check directive: Nginx proactively sends probe requests to upstream servers at interval. Parameters: interval, fails (mark unhealthy after N failures), passes (mark healthy after N successes), uri (probe path), match (custom response validation: status 200-299, header contains, body matches regex). Open-source only has passive checks (fail_timeout/max_fails). For open-source, lua-resty-upstream-healthcheck or Kubernetes readiness probes provide equivalent functionality.

Q45. Explain how Nginx implements access control lists.

allow/deny directives: allow 10.0.0.0/8; deny all; — evaluated top-to-bottom, first match wins. geo module for CIDR-based variables. auth_basic for HTTP Basic Auth. auth_request for subrequest-based auth (validate JWT by calling auth microservice). limit_except restricts methods: limit_except GET POST { deny all; }. For Java APIs, auth_request to a Spring Security validation endpoint is a common pattern.

Q46. How does Nginx handle IPv6?

listen [::]:80 ipv6only=on; for IPv6-only. listen 0.0.0.0:80; listen [::]:80; for dual-stack. On most Linux systems, listen [::]:80 with ipv6only=off handles both IPv4 and IPv6 via IPv4-mapped IPv6 addresses. \$remote_addr contains the actual client IP (IPv4 or IPv6). limit_req_zone uses \$binary_remote_addr (16 bytes for IPv6 vs 4 for IPv4) for correct per-IP rate limiting.

Q47. What is the nginx_status active connections breakdown?

Active connections = Reading + Writing + Waiting. Reading: connections where Nginx is reading request headers. Writing: connections where Nginx is sending response (active processing). Waiting: keepalive connections idle between requests (connection held open, no data transfer). High Waiting: clients reusing keepalive connections (good). High Reading: slow clients sending large headers. High Writing: slow clients or slow upstream responses.

Q48. How do you tune Nginx for a Java microservices deployment?

worker_processes auto; worker_connections 4096; keepalive 32 (upstream keepalive pool); proxy_http_version 1.1; proxy_set_header Connection ""; (enable upstream keepalive); proxy_connect_timeout 5s; proxy_read_timeout 60s; proxy_send_timeout 60s; upstream_response_time in access_log; proxy_next_upstream error timeout; (retry on failure); limit_req for DDoS protection; gzip on for JSON APIs; open_file_cache for static assets.

Q49. What is the difference between proxy_pass with and without trailing slash?

proxy_pass http://backend: URI is passed unchanged (including location prefix). proxy_pass http://backend/: replaces the location prefix with / (strips the matched prefix). Example: location /api/ { proxy_pass http://backend/; } — /api/users becomes /users on backend. location /api/ { proxy_pass http://backend; } — /api/users becomes /api/users on backend. Critical distinction for Java API routing.

Q50. How does Nginx implement connection limiting per IP?

limit_conn_zone \$binary_remote_addr zone=perip:10m; defines shared memory tracking connections per IP. limit_conn perip 20; limits to 20 simultaneous connections. limit_conn_log_level error; sets log level for exceeded limits. limit_conn_status 503; response code. Complements limit_req (request rate) for comprehensive abuse protection. Combined: limit_conn prevents connection exhaustion, limit_req prevents request flood.

Q51. Explain Nginx's handling of chunked transfer encoding.

HTTP/1.1 chunked encoding allows streaming responses without Content-Length. Nginx by default buffers chunked responses and re-chunks or sets Content-Length. proxy_buffering off disables this for streaming. chunked_transfer_encoding on (default) enables chunked responses to clients. For Spring Boot streaming endpoints (Flux/SSE), ensure proxy_buffering off and proxy_read_timeout is sufficiently long.

Q52. How do you configure Nginx for gRPC proxying?

grpc_pass grpc://backend:50051; for TLS gRPC. grpc_pass grpc://backend:50051; for plaintext. Requires http2 on server block (listen 443 ssl http2). grpc_set_header Host \$host; grpc_read_timeout 300s; grpc_send_timeout 300s. grpc_ssl_certificate/grpc_ssl_certificate_key for mTLS to backend. Nginx terminates gRPC from clients and proxies to Java gRPC backends — enables gRPC-to-gRPC with TLS termination.

Q53. What is the Nginx variable \$upstream_response_time and how is it used for observability?

\$upstream_response_time: time in seconds (ms resolution) for upstream to respond. \$upstream_connect_time: time to establish upstream connection. \$upstream_header_time: time to receive response headers. Include in access_log JSON format for P50/P95/P99 latency analysis of Java backends in ELK/Grafana. Alert when p95 \$upstream_response_time exceeds SLA. Distinguish Nginx processing time (\$request_time - \$upstream_response_time) from backend time.